



Computer Networks | Report HW 1

Васюков Александр Владимирович
БПИ-235
avvasiukov@edu.hse.ru

Цель работы: научиться измерять пропускную способность компьютерной сети в зависимости от разных параметров передачи.

Я претендую на оценку до 10 баллов, генеративный ИИ не был использован для работы над этим заданием.

Оборудование и сеть

- Сервер запускался на удаленной виртуальной машине (VM) от преподавателя, IP: `178.154.195.170`
- Клиент запускался в локальной сети, IP: `192.168.1.25`

Реализация

Клиент и сервер реализованы на Java с использованием `Socket` и `ServerSocket`.

Клиент каждый раз при отсылке формирует массив из случайного набора байт и перед его отсылкой дожидается ответа от сервера. Клиент замеряет время, которое требуется для того, чтобы отправить массив байт и дождаться ответа от сервера.

Сервер в ответ возвращает строку с текущей датой/временем формата

`YYYY.MM.DD HH:MM:SS`.

Измерение времени проводилось так: перед отправкой конкретного запроса (вызов `writeInt(len)`, `write(payload)`, `flush()`) клиент фиксировал `t0 = System.currentTimeMillis()`. После получения ответа сервера (вызов `readLine()`) фиксировался `t1 = System.currentTimeMillis()`. Время запроса равно `t1 - t0`. Для каждой итерации `K` выполнялись `Q` повторов. В логах сохранены строки прогресса с усреднёнными значениями, а полные данные экспортированы в CSV для построения графиков.

Для каждой итерации `K` выполняется `Q` под-итераций; среднее время для `K = sum(Q)/Q`. Результаты записываются в CSV-файлы `results<#>.csv`.

Wireshark

Скриншот Wireshark с фильтром `ip.addr == 178.154.195.170 && tcp.port == 10000`

No.	Time	Source	Destination	Protocol	Length	Info
653	24.310547	192.168.1.25	178.154.195.170	TCP	78	59029 → 10000 [SYN, ECE, CWR] Seq=0 Win=0
654	24.332776	178.154.195.170	192.168.1.25	TCP	54	10000 → 59029 [RST, ACK] Seq=1 Ack=1 Win=0
1801	33.039122	192.168.1.25	178.154.195.170	TCP	78	59030 → 10000 [SYN, ECE, CWR] Seq=0 Win=0
1802	33.055643	178.154.195.170	192.168.1.25	TCP	74	10000 → 59030 [SYN, ACK, ECE] Seq=0 Ack=1
1803	33.055803	192.168.1.25	178.154.195.170	TCP	66	59030 → 10000 [ACK] Seq=1 Ack=1 Win=131
1804	33.060158	192.168.1.25	178.154.195.170	TCP	70	59030 → 10000 [PSH, ACK] Seq=1 Ack=1 Win=131
1805	33.060200	192.168.1.25	178.154.195.170	NDMP	74	[NDMP fragment]
1806	33.077407	178.154.195.170	192.168.1.25	TCP	66	10000 → 59030 [ACK] Seq=1 Ack=5 Win=652
1807	33.077409	178.154.195.170	192.168.1.25	TCP	66	10000 → 59030 [ACK] Seq=1 Ack=13 Win=652
1809	33.109833	178.154.195.170	192.168.1.25	TCP	86	10000 → 59030 [PSH, ACK] Seq=1 Ack=13 Win=652
1810	33.109933	192.168.1.25	178.154.195.170	TCP	66	59030 → 10000 [ACK] Seq=13 Ack=21 Win=1
1811	33.110228	192.168.1.25	178.154.195.170	TCP	70	59030 → 10000 [PSH, ACK] Seq=13 Ack=21 Win=1
1812	33.110253	192.168.1.25	178.154.195.170	NDMP	74	[NDMP fragment]
1814	33.124244	178.154.195.170	192.168.1.25	TCP	66	10000 → 59030 [ACK] Seq=21 Ack=25 Win=6
1815	33.124246	178.154.195.170	192.168.1.25	TCP	86	10000 → 59030 [PSH, ACK] Seq=21 Ack=25 Win=6
1816	33.124382	192.168.1.25	178.154.195.170	TCP	66	59030 → 10000 [ACK] Seq=25 Ack=41 Win=1
1817	33.124517	192.168.1.25	178.154.195.170	TCP	70	59030 → 10000 [PSH, ACK] Seq=25 Ack=41 Win=1
1818	33.124543	192.168.1.25	178.154.195.170	NDMP	74	[NDMP fragment]
1819	33.140841	178.154.195.170	192.168.1.25	TCP	66	10000 → 59030 [ACK] Seq=41 Ack=37 Win=6
1820	33.140844	178.154.195.170	192.168.1.25	TCP	86	10000 → 59030 [PSH, ACK] Seq=41 Ack=37 Win=6
1821	33.140927	192.168.1.25	178.154.195.170	TCP	66	59030 → 10000 [ACK] Seq=37 Ack=61 Win=1
1822	33.141068	192.168.1.25	178.154.195.170	TCP	70	59030 → 10000 [PSH, ACK] Seq=37 Ack=61 Win=1
1823	33.141093	192.168.1.25	178.154.195.170	NDMP	74	[NDMP fragment]
1824	33.157456	178.154.195.170	192.168.1.25	TCP	66	10000 → 59030 [ACK] Seq=61 Ack=49 Win=6

Эксперименты

Параметры экспериментов

Пробовал малые и большие N, а также различные Q — чтобы исследовать влияние размера сообщения и статистики повторов на усреднённое время.

В экспериментах использовался флаг `TCP_NODELAY` через вызов `socket.setTcpNoDelay(...)`. Включение `tcpNoDelay=true` отключает алгоритм Nagle и заставляет стек отправлять мелкие сегменты без их агрегирования, что уменьшает задержку для мелких сообщений. Установка `tcpNoDelay=false` включает Nagle и у принимающей стороны может приводить к увеличению задержки. Для демонстрации этого эффекта выполнены эксперименты с обоими значениями флага.

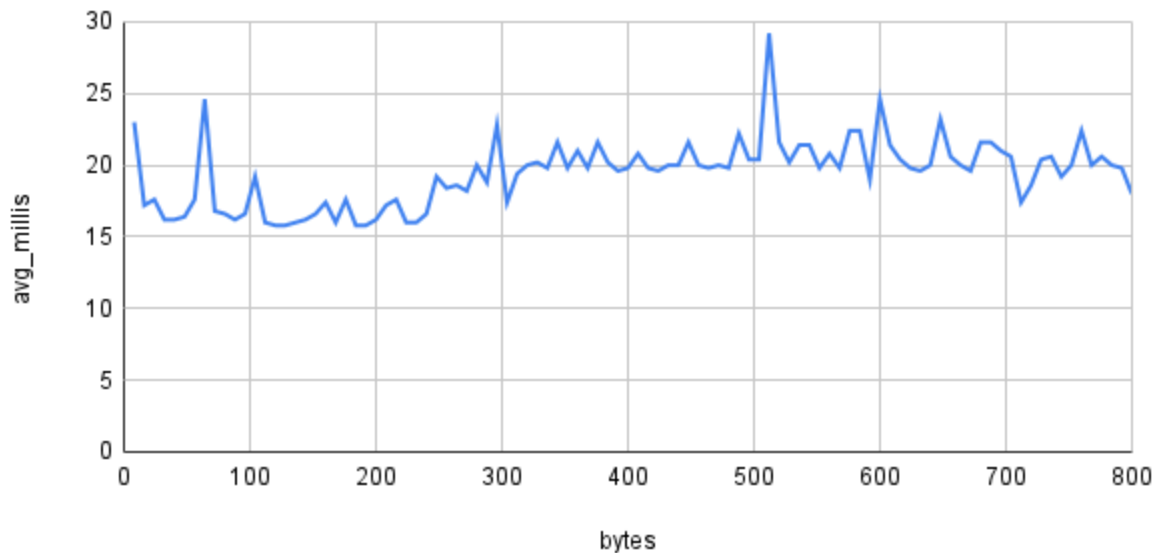
М были использованы меньше, чем требовалось изначально в задании, потому что иначе требовалось долго передавать большие данные, что заняло бы несколько часов. Поэтому в замерах использовались более маленькие значения

Эксперимент 1

```
alexvasyukov@Alexanders-MacBook-Air ClientServerApp % java -jar client.jar 178.154.195.170 10000 8 100 5 true results1
sv
Client: 178.154.195.170:10000, N=8, M=100, Q=5, tcpNoDelay=true
Progress: K=0/99      len=8    avg=23.0000ms
Progress: K=10/99     len=88   avg=16.2000ms
Progress: K=20/99     len=168  avg=16.0000ms
Progress: K=30/99     len=248  avg=19.2000ms
Progress: K=40/99     len=328  avg=20.2000ms
Progress: K=50/99     len=408  avg=20.8000ms
Progress: K=60/99     len=488  avg=22.2000ms
Progress: K=70/99     len=568  avg=19.8000ms
Progress: K=80/99     len=648  avg=23.2000ms
Progress: K=90/99     len=728  avg=20.4000ms
```

Experiment 1

N=8, M=100, Q=5, tcpNoDelay=true



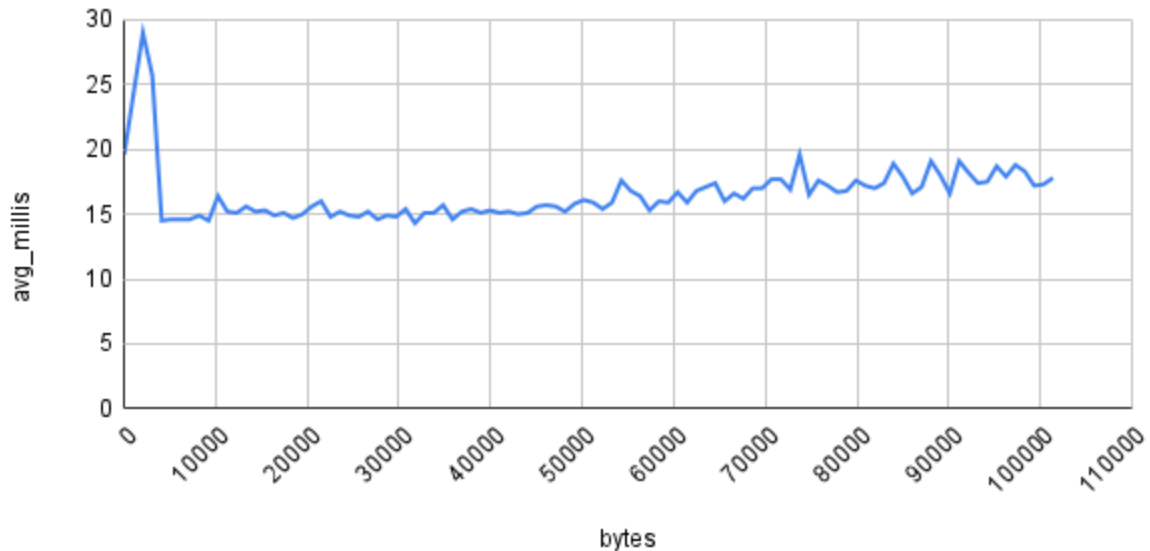
Значения сначала находятся в пределах 16 миллисекунд и примерно с середины вырастают до 20 миллисекунд. График сильно колеблется.

Эксперимент 2

```
alexvasyukov@Alexanders-MacBook-Air ClientServerApp % java -jar client.jar 1
.csv
Client: 178.154.195.170:10000, N=1024, M=100, Q=10, tcpNoDelay=true
Progress: K=0/99      len=8      avg=19.6000ms
Progress: K=10/99     len=10248   avg=16.4000ms
Progress: K=20/99     len=20488   avg=15.6000ms
Progress: K=30/99     len=30728   avg=15.4000ms
Progress: K=40/99     len=40968   avg=15.1000ms
Progress: K=50/99     len=51208   avg=15.9000ms
Progress: K=60/99     len=61448   avg=15.9000ms
Progress: K=70/99     len=71688   avg=17.7000ms
Progress: K=80/99     len=81928   avg=17.0000ms
Progress: K=90/99     len=92168   avg=18.2000ms
Results written to results2.csv
```

Experiment 2

N=1024, M=100, Q=10, tcpNoDelay=true



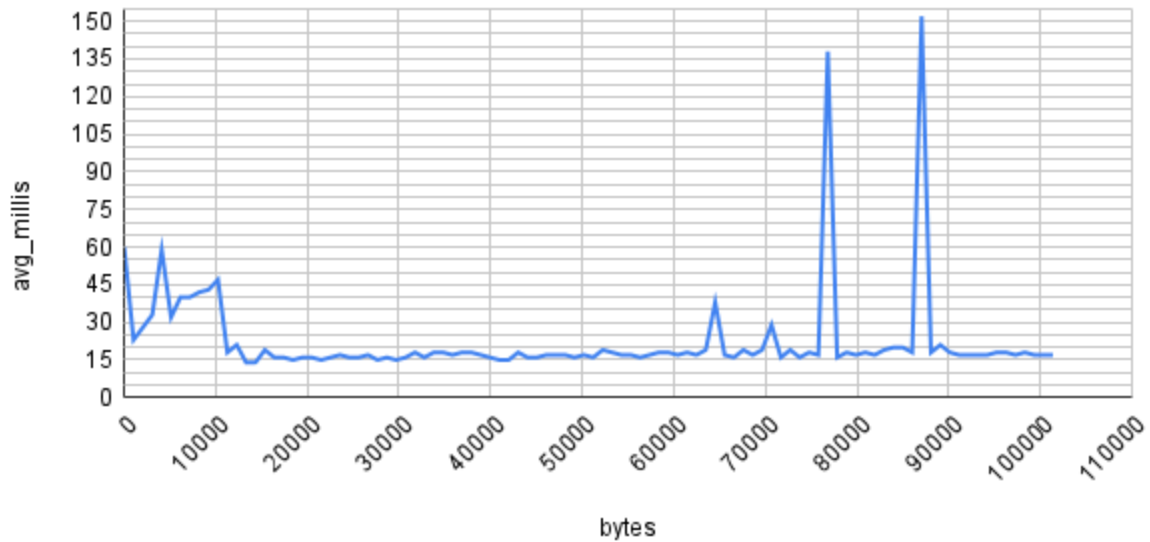
В начале замера есть резкий пик до 30 миллисекунд, но дальше график стабилизируется на отметке в 15 миллисекунд и лишь в конце немного вырастает до 17.

Эксперимент 3

```
alexvasyukov@Alexanders-MacBook-Air ClientServerApp % java -jar client.jar 178.154.195.170
csv
Client: 178.154.195.170:10000, N=1024, M=100, Q=1, tcpNoDelay=true
Progress: K=0/99      len=8      avg=60.0000ms
Progress: K=10/99     len=10248   avg=47.0000ms
Progress: K=20/99     len=20488   avg=16.0000ms
Progress: K=30/99     len=30728   avg=16.0000ms
Progress: K=40/99     len=40968   avg=15.0000ms
Progress: K=50/99     len=51208   avg=16.0000ms
Progress: K=60/99     len=61448   avg=18.0000ms
Progress: K=70/99     len=71688   avg=16.0000ms
Progress: K=80/99     len=81928   avg=17.0000ms
Progress: K=90/99     len=92168   avg=17.0000ms
Results written to results3.csv
```

Experiment 3

N=1024, M=100, Q=1, tcpNoDelay=true



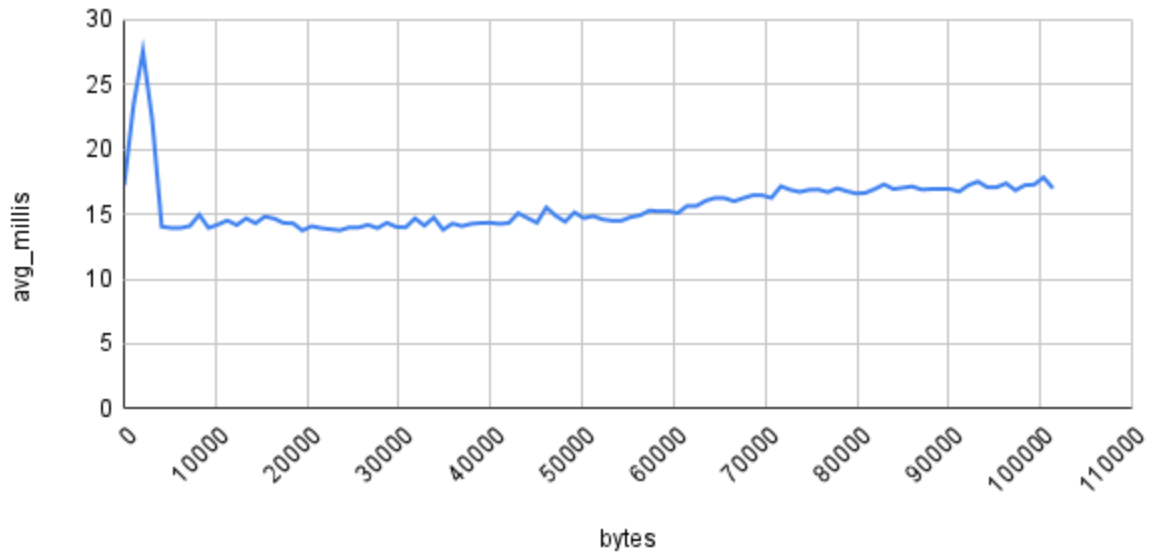
Опять же в начале видны резкие всплески, но потом всё стабилизируется около 15 миллисекунд. В конце есть 2 явных пика, достигающих до 135 и 150 миллисекунд.

Эксперимент 4

```
alexvasyukov@Alexanders-MacBook-Air ClientServerApp % java -jar client.jar 178.154.195.170
.csv
Client: 178.154.195.170:10000, N=1024, M=100, Q=50, tcpNoDelay=true
Progress: K=0/99      len=8      avg=17.2400ms
Progress: K=10/99     len=10248   avg=14.2000ms
Progress: K=20/99     len=20488   avg=14.0800ms
Progress: K=30/99     len=30728   avg=13.9800ms
Progress: K=40/99     len=40968   avg=14.2600ms
Progress: K=50/99     len=51208   avg=14.8600ms
Progress: K=60/99     len=61448   avg=15.6400ms
Progress: K=70/99     len=71688   avg=17.1600ms
Progress: K=80/99     len=81928   avg=16.9400ms
Progress: K=90/99     len=92168   avg=17.2400ms
Results written to results4.csv
```

Experiment 4

N=1024, M=100, Q=50, tcpNoDelay=true



Снова в начале есть выброс, но дальше значения стабилизируются и немного растут.

Эксперимент 5

```
alexvasyukov@Alexanders-MacBook-Air ClientServerApp % java -jar client.jar 178.154.195.170
v
Client: 178.154.195.170:10000, N=10, M=100, Q=5, tcpNoDelay=true
Progress: K=0/99      len=8   avg=20.4000ms
Progress: K=10/99     len=108 avg=16.0000ms
Progress: K=20/99     len=208 avg=16.6000ms
Progress: K=30/99     len=308 avg=19.8000ms
Progress: K=40/99     len=408 avg=18.8000ms
Progress: K=50/99     len=508 avg=16.2000ms
Progress: K=60/99     len=608 avg=21.6000ms
Progress: K=70/99     len=708 avg=19.8000ms
Progress: K=80/99     len=808 avg=20.4000ms
Progress: K=90/99     len=908 avg=20.8000ms
Results written to results5.csv
```

Experiment 5

N=10, M=100, Q=5, tcpNoDelay=true



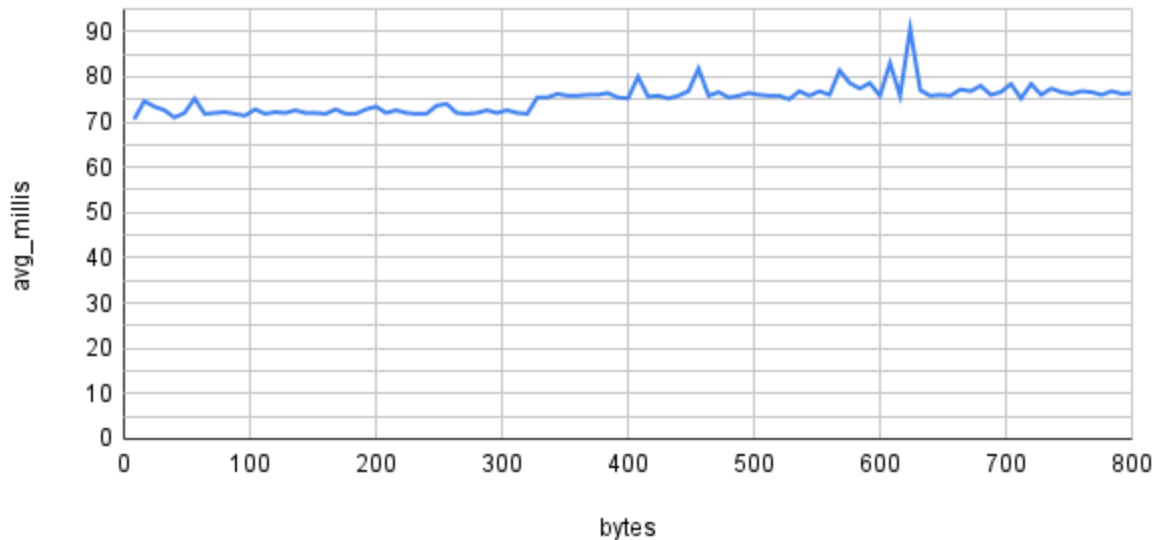
Теперь график в целом стабильный, около 20 миллисекунд, но есть один выброс до 75 миллисекунд.

Эксперимент 6

```
alexvasyukov@Alexanders-MacBook-Air ClientServerApp % java -jar client.jar 178.154.195.170
v
Client: 178.154.195.170:10000, N=8, M=100, Q=5, tcpNoDelay=false
Progress: K=0/99      len=8   avg=70.6000ms
Progress: K=10/99     len=88  avg=71.8000ms
Progress: K=20/99     len=168 avg=72.8000ms
Progress: K=30/99     len=248 avg=73.6000ms
Progress: K=40/99     len=328 avg=75.4000ms
Progress: K=50/99     len=408 avg=80.0000ms
Progress: K=60/99     len=488 avg=75.8000ms
Progress: K=70/99     len=568 avg=81.4000ms
Progress: K=80/99     len=648 avg=76.0000ms
Progress: K=90/99     len=728 avg=76.0000ms
Results written to results6.csv
```


Experiment 6

N=8, M=100, Q=5, tcpNoDelay=false



Значения выросли почти в 2-3 раза по сравнению с прошлыми экспериментами.

Интерпретация графиков

- В большинстве экспериментов (1, 2, 3, 4, 5) среднее время запроса практически не растёт при увеличении объёма передаваемых данных. Это означает, что передача каждого пакета находится в режиме, где основную долю задержки создаёт RTT (скорость сети и ОС, приложение), а не пропускная способность канала: время передачи сильно мало по сравнению с нагрузкой за отправку и получение.
- При `Q=1` (эксперимент 3) наблюдается большая дисперсия и отдельные выбросы (например, 60 миллисекунд) — одиночные измерения ненадёжны. При увеличении `Q` (например, `Q=50` в эксперименте 4) среднее значение становится стабильным и шум сглаживается.
- При `tcpNoDelay=false` (эксперимент 6) среднее время значительно выросло (70-80 миллисекунд). Это проявление совместного действия алгоритма

Nagle и delayed-ACK: малые сегменты буферизуются на отправляющей стороне и ждут подтверждения, что увеличивает задержку.

- Высокие значения в первых итерациях могут объясняться несколькими причинами: компиляция, разогрев JVM, первоначальная активность GC или кратковременные сетевые случайности.

Наблюдения и выводы

- В рассматриваемых условиях (локальная машина и VM в облаке) основную долю времени запроса составляет фиксированная сетевая задержка RTT и системные накладные расходы — рост объёма полезной нагрузки не оказывает значительного влияния на усреднённое время запроса.
- Параметр `Q` критически влияет на статистическую устойчивость: `Q=1` даёт нестабильные и легко поддающиеся выбросам оценки.
- Отключение алгоритма Nagle (`tcpNoDelay=true`) существенно уменьшает задержки для мелких сообщений; при `tcpNoDelay=false` наблюдается многократное увеличение среднего времени.